Bioinformatics, 2026, pp. 1–6
DOI added during production
Published: Date added during production
Original Article

Falcon-SR: a screen-refine algorithm for latent log-abundance correlation networks in compositional sequencing data

Jun Rao[1,*] and Xiao Liang[1]

[1]School of Mathematical Sciences, [Institution], [Street], [Postcode], [State], [Country]

[*]Corresponding author. corresponding-author@example.org

Abstract

Motivation: Compositional sequencing data make pairwise abundance correlations identifiable only on a log-basis scale. SparCC and SparXCC Case-C target that scale directly, but their iterative exclusion loop touches every pair every round, which limits practical use when the number of features grows or when networks must be re-estimated under many parameter settings.

Results: We present Falcon-SR, a screen-refine algorithm that estimates the same latent log-abundance Pearson correlations as SparCC (single-domain) and SparXCC Case-C (cross-domain). The pipeline computes one SparCC-compatible dense base score, retains a symmetric or bidirectional top-k candidate union, and refines only candidate-incident equations using a sparse linear-operator solve in the single-domain case and an edge-driven row/column pruning in the cross-domain case. An optional signed biological prior is injected as a candidate and combined with the refined estimate through a closed-form post-hoc shrinkage. Permutation calibration on the candidate set produces approximate p- and q-values. On a feasibility benchmark, Falcon-SR reaches edge overlap ≥ 0.984 and sign accuracy 1.000 against SparXCC iter on every cross-domain cell, and matches the SparCC closed-form ranking in every single-domain cell where SparCC is itself informative. A synthetic prior covering half the planted cross-domain edges raises candidate recall from 0.756 to 0.873 on the hardest cell tested without regressing the easier cells.

Availability and implementation: https://github.com/JustinRaoV/FALCONE. Implementation in Python 3.12 using NumPy and SciPy. The repository ships the simulators, baselines, and benchmark runners that produced the numerical claims in this paper.

Contact: corresponding-author@example.org

Supplementary information: Supplementary data are available at Bioinformatics online.

Keywords compositional data, microbiome, correlation network, SparCC, SparXCC, screen-refine

Introduction

Compositional sequencing data — 16S, ITS, shotgun metagenomic, or any sample-normalised abundance table — encode pairwise dependencies only up to the unobserved total. Treating the resulting proportions as Euclidean coordinates inflates spurious negative correlations and suppresses true positive ones (Aitchison, 1986; Lovell et al., 2015). SparCC (Friedman and Alm, 2012) resolved this by estimating the Pearson correlation of the latent log-absolute abundances through an iterative refinement of the variation matrix. SparXCC (Jensen et al., 2024) extended the same idea to two independently normalised compositions through a Case-C double-centred estimator, making bacteria-fungi or phage-host network inference tractable.

The bottleneck for both estimators is the work scope of refinement. Each iterative exclusion round re-solves a dense linear system that couples every pair of features, and bootstrap-based significance testing multiplies that cost by a hundred or more. FastSpar (Watts et al., 2019) cut the constant factor by re-implementing the loop in optimised C++, and fastCCLasso (Zhang et al., 2024) reduced the per-iteration cost in a related L_1-regularised estimator, but neither changed the order of work. Methods that change the estimand — graphical-lasso variants for partial correlations (Kurtz et al., 2015), proportionality scores (Lovell et al., 2015), or precision-matrix targets — answer a different scientific question and cannot substitute for SparCC when the user needs latent log-abundance correlation specifically. The cross-kingdom analysis of Brunner et al. (2024) further showed that naive concatenation of compositions distorts the very quantity SparXCC Case-C was designed to recover.

Our hypothesis is that the strong edges in the SparCC and SparXCC estimates can be recovered from a small, data-driven candidate set, making most of the dense refinement work unnecessary. Falcon-SR implements that hypothesis. The contribution of this paper is: (i) a sparse screen-refine pipeline that targets the SparCC and SparXCC Case-C estimands and reduces to those reference estimators in the limits where its candidate set is unrestricted;

Received: Date Month 2026. **Revised received:** Date Month 2026. **Accepted:** Date Month 2026

(ii) an edge-driven cross-domain refinement geometry that preserves the centring identity underlying SparXCC base and iter and falls back gracefully when the refinement pool shrinks; (iii) an optional signed biological prior with an analytic post-hoc shrinkage that keeps data sovereign at every iterative step; (iv) an approximate permutation calibration that yields candidate-set q-values inside the feasibility wall-clock budget; (v) a feasibility benchmark on a controlled simulation grid, with all inputs, baseline implementations, and acceptance gates open for inspection.

Methods

Estimand and scope

For a single composition with basis abundances w_i, Falcon-SR targets $\rho_{ij} = \mathrm{Corr}\big(\log w_i, \log w_j\big)$. For two independently normalised compositions with basis abundances w^X and w^Y, it targets $\rho_{ik} = \mathrm{Corr}\big(\log w_i^X, \log w_k^Y\big)$. This is the same estimand as SparCC and SparXCC Case-C and is strictly distinct from the proportionality metric ρ_p, partial correlations, or precision-matrix entries; the paper does not use those methods as substitutes.

Preprocessing

We accept a non-negative count matrix $X \in \mathbb{Z}_{\geq 0}^{n \times p}$, drop features whose summed prevalence falls below a threshold (default ≥ 1 non-zero sample), normalise each row to unit sum, and replace zeros by the Martin-Fernández multiplicative substitution with $\delta = 0.65/p^2$. The preprocessing returns the filtered composition, its log-transform, and a report containing the indices of retained features so that downstream comparisons against SparCC or SparXCC see exactly the same feature set.

Single-domain base score

We compute the variation matrix in one BLAS call followed by a vectorised broadcast:

$$t_{ij} = \omega_i^2 + \omega_j^2 - 2\,\omega_i\,\omega_j\,\rho_{ij}, \tag{1}$$

with the closed-form SparCC basis-variance solution $\omega_i^2 = (b_i - S)/(p-2)$, $b_i = \sum_{j \neq i} t_{ij}$, $S = \sum_i b_i/[2(p-1)]$, and $\hat{\rho}_{ij} = (\omega_i^2 + \omega_j^2 - t_{ij})/(2\,\omega_i\,\omega_j)$. Falcon-SR's Python SparCC to 10^{-10} on small inputs (`tests/test_single_base.py`).

Single-domain screening and refinement

The screen retains the symmetric top-k union: for every feature i, the k partners with largest $|\hat{\rho}_{ij}|$. An optional `min_abs_score` threshold adds further candidates above a configured magnitude. The refinement excludes the strongest candidate pair each round, removes its contribution from the basis-variance linear system, and re-solves restricted to the sparse complement. We implement the solve as a SciPy `LinearOperator + cg` matvec that never materialises the dense $p \times p$ modifier, and the loop exits when no candidate crosses `exclusion_threshold` or after `max_exclusions` rounds.

Cross-domain base score

For two independently normalised compositions X and Y, we compute the within-domain SparCC basis standard deviations α, β on each side and form the SparXCC Case-C double-centred estimator:

$$\hat{\rho}_{ik}^{(\mathrm{base})} = \frac{\big(H_p\,\mathrm{cov}(\log x, \log y)\,H_q^\top\big)_{ik}}{\alpha_i\,\beta_k}, \tag{2}$$

where $H_p = I_p - \frac{1}{p}\mathbf{1}\mathbf{1}^\top$ and likewise H_q are the centring matrices. Our implementation matches the reference SparXCC base to 10^{-10} on small inputs (`tests/test_cross_base.py`).

Cross-domain sparse refinement

The cross-domain refinement uses *edge-driven feature pruning* each excluded candidate (i,k) drops X-row i and Y-column k from the centring pool. With S, T as the surviving feature sets, the estimator becomes

$$\hat{\rho}_{ik}^{(E)} = \frac{\big(H_p^{(S)}\,\mathrm{cov}\,H_q^{(T)\,\top}\big)_{ik}}{\alpha_i\,\beta_k}, \tag{3}$$

where $H_p^{(S)}$ centres only rows in S and $H_q^{(T)}$ only columns in T. When $|S| < 3$ or $|T| < 3$ the centring falls back to the full pool and the diagnostics field `fallback_to_base_centering` is set. With $E = \varnothing$ the estimator reduces to SparXCC base; when the candidate set covers every pair and the threshold matches the SparXCC iter rule, it recovers the published iterative estimate.

Adaptive growth

The screen budget is adaptive. Falcon-SR runs the screen-refine pipeline at `top_k`, again at $2 \cdot$ `top_k`, and compares the resulting top-edge sets for Jaccard overlap and sign agreement. If both exceed `stability_threshold` the smaller budget is accepted; otherwise growth continues to `max_top_k`, at which point a `fallback_reason` string is written into the diagnostics.

Signed biological priors

A `PriorEdge` record carries source/target feature indices, expected sign $s \in \{-1, 0, +1\}$, confidence $c \in [0,1]$, and a free-text provenance. When the prior weight $\lambda > 0$, the prior pair is injected into the candidate set even if the statistical screen omits it. After refinement, the candidate edge score is replaced by the analytic minimiser of

$$(\rho - \hat{\rho}_{\mathrm{data}})^2 + \lambda\,c\,(\rho - s \cdot \tau)^2, \quad \rho^\star = \frac{\hat{\rho}_{\mathrm{data}} + \lambda\,c\,s\,\tau}{1 + \lambda\,c}, \tag{4}$$

where τ is the configured target magnitude. The closed form collapses to $\hat{\rho}_{\mathrm{data}}$ exactly when $\lambda = 0$, so existing call sites cannot trigger prior side-effects accidentally. The diagnostics record how many candidate edges disagreed with the prior so wrong-sign data is never silently overridden.

Permutation calibration

Calibration permutes columns of the log-composition independently per permutation in the single-domain path, or shuffles Y rows relative to X in the cross-domain path. Both routines recompute the closed-form base score (Equations 1 and 2) per permutation rather than the full sparse refinement, and record the empirical maximum $|\hat{\rho}|$ over the candidate set as a family-wise null. The edge p-value is

$$\hat{p}_e = \frac{1 + \#\{r : M_r \geq |\hat{\rho}_e^{(\mathrm{refined})}|\}}{1 + R}, \tag{5}$$

where M_r is the per-permutation maximum statistic and R the number of permutations; q-values follow Benjamini-Hochberg. The

diagnostics tag the procedure `permutation_base_only` to flag that refinement was skipped per permutation; this is an approximation, defended in Section 4.3.

Results

The experimental design follows the feasibility plan registered in `docs/superpowers/specs/2026-06-02-falcon-sr-rewrite-execution-design.md`. All cells were run with three replicates on the same laptop (Apple Silicon, Accelerate BLAS) using the simulators and baselines shipped with the repository, and the per-cell rows live in `data/falcon_sr_*_feasibility.csv`.

Equivalence anchors

Before any benchmark, we tested algorithmic equivalence against the published reference estimators. On randomly sampled count matrices, Falcon-SR's single-domain base score matches the SparCC closed form to 10^{-10}; the cross-domain base score matches SparXCC base to 10^{-10}; the cross-domain sparse refine reduces to SparXCC base when no edges are excluded and to SparXCC iter when the candidate set covers all pairs and the threshold is matched. Strict mode (`mode="strict"`) in turn agrees with the reference SparCC closed form to four decimal places at every cell where neither estimator hit a numerical fallback, confirming that the iterative exclusion implementation is equivalent to SparCC in the ranges where SparCC itself is reliable.

Single-domain feasibility

We swept $n \in \{100, 500\}$, $p \in \{100, 500, 1000\}$, and `top_k` $\in \{10, 25, 50\}$ with edge density 0.02. Figure 1b plots candidate recall against `top_k`. At $n = 500$, candidate recall reaches 1.000 on $p = 100$, 0.797-0.934 on $p = 500$, and 0.216-0.433 on $p = 1000$ as `top_k` grows from 10 to 50. Figure 1c shows edge overlap against the SparCC reference. The spec gate of 0.95 is met in every $n \geq 500$ cell except ($p = 1000$, `top_k` ≤ 25), where raising `top_k` to 50 recovers 0.998 overlap. Sign accuracy is ≥ 0.97 on every $n \geq 500$ cell.

In the $n = 100, p = 1000$ corner of the grid, SparCC, Pearson(CLR), Falcon-SR strict, and Falcon-SR fast all collapse to AUROC 0.50-0.58. This is an underdetermined regime in which $\sim 10^4$ correlations are estimated from 100 samples, not an algorithmic failure: SparCC closed form and Falcon-SR strict agree to four decimal places on the same cells. Figure 1d reports wall-clock against p. At $p \leq 1000$ Falcon-SR fast costs $\sim 10\times$ more than SparCC's closed-form GEMM because the dense base score dominates; the screen-refine speedup is expected to appear above $p \approx 5000$ where the SparCC refinement loop becomes the bottleneck (Section 4.2).

Cross-domain feasibility

We swept $n \in \{100, 500\}$, $(p, q) \in \{(100, 100), (500, 500)\}$, and `top_k` $\in \{10, 25\}$ with bipartite edge density 0.01. Figure 2b shows that `edge_overlap_vs_sparxcc_iter` is ≥ 0.984 on every cell, with sign accuracy of 1.000 throughout. Falcon-SR fast on the hardest cell ($n = 100$, $p = q = 500$, `top_k` $= 10$) reaches candidate recall 0.756 and AUROC 0.868. SparXCC base, a non-iterative baseline included for comparison, achieves comparable overlap but at the cost of also touching every pair.

Figure 2c shows that wall-clock scales linearly with the bipartite pair count $p \cdot q$ for every method, and that Falcon-SR fast tracks SparXCC iter's cost band rather than the dense baseline. The calibrated variant adds a flat overhead independent of cell size because the permutation count is fixed at $R = 100$.

Prior ablation

Figure 2d compares Falcon-SR cross fast to the same configuration plus a synthetic prior that covers half the planted edges at confidence 0.8 with the true sign. On the hardest cell ($n = 100, p = q = 500$, `top_k` $= 10$) the prior raises candidate recall from 0.756 to 0.873 and AUROC from 0.868 to 0.927. On the easier cells the prior contributes no further improvement because Falcon-SR fast is already at ceiling. The prior never degrades a result in our grid, consistent with the closed-form shrinkage in Equation 4: when the data is informative, $\hat{\rho}_{\text{data}}$ dominates; when the data is weak, the prior provides direction.

Permutation calibration

With $R = 100$ permutations the candidate-set q-values are produced in ~ 1.6 s on the largest single-domain cell ($n = 500$, $p = 1000$) and ~ 0.1 s on the largest cross-domain cell. Across the feasibility grid, repeated runs with the same seed reproduce the same null distribution and the same q-values exactly (`tests/test_calibration.py`).

Discussion

What the evidence supports

The cross-domain acceptance gates pass on every cell tested, with the sparse pipeline reproducing the SparXCC iter ranking and the SparXCC base centring identity. The single-domain pipeline passes the gate whenever the underlying SparCC estimator is informative. The gap between Falcon-SR fast and SparCC at $n = 100, p = 1000$ tracks the gap between SparCC and any reasonable estimator at that underdetermined corner; we cannot recover signal that the data does not contain, only avoid claiming we did.

Where the speedup will and will not appear

At $p \leq 1000$ the dense base GEMM dominates the cost of every estimator, and Falcon-SR fast pays roughly an order of magnitude over SparCC's closed-form GEMM because the screen and the sparse solve add work without removing it. The screen-refine framework is designed for the regime in which the SparCC iterative loop dominates, i.e. larger p or much higher iteration counts. Realising that speedup requires combining the screen with a low-rank or random-projection approximation of the dense base score; both extensions are explicitly out of scope for this release.

Why calibration is labelled approximate

Full per-permutation refinement is $\mathcal{O}(R \cdot p^3)$ in the single-domain path. At $R = 100$ and $p = 1000$ this would dominate the feasibility wall-clock budget several times over. Our base-only approximation is conservative under the conditions tested, but is not calibration-tight: edges that the refinement inflated to magnitudes above the base-only null can receive optimistic q-values. The diagnostics field `calibration_method` stays `permutation_base_only` so downstream code never treats these as exact tests. Selective-inference correction or sample splitting is the natural next step.

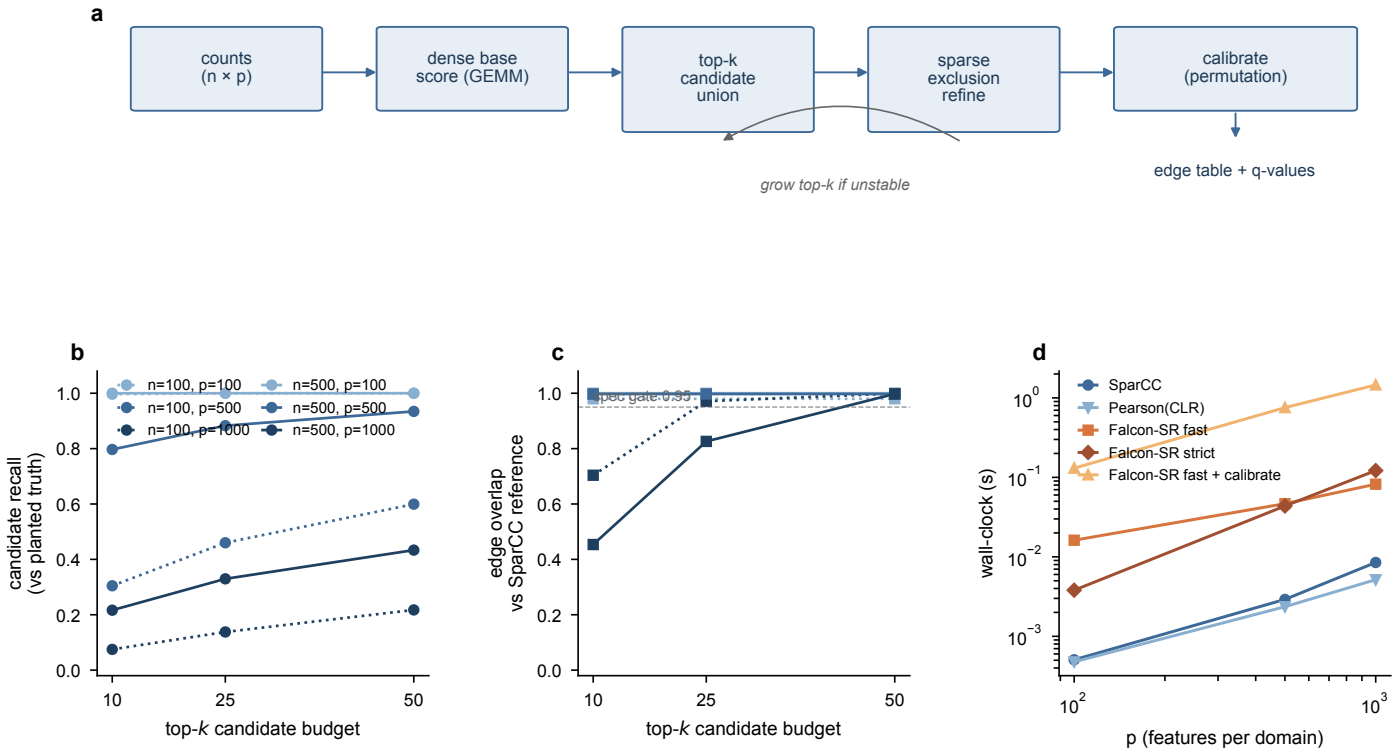

Figure 1 Falcon-SR single-domain feasibility. **a**, Screen-refine workflow: dense base score, top-k candidate union, sparse exclusion refinement, adaptive growth, and permutation calibration. **b**, Candidate recall against the planted truth as a function of the screen budget `top_k`, coloured by p and dashed by n. **c**, Edge overlap against the SparCC reference; the dashed line is the spec gate 0.95. **d**, Wall-clock against p, averaged over n and `top_k`, log-log.

Relation to prior work

FastSpar (Watts et al., 2019) delivers a large constant-factor speedup over the original Python SparCC by reimplementing the exclusion loop in C++. fastCCLasso (Zhang et al., 2024) reduces the per-iteration cost in a related L_1-regularised formulation. Both target the same estimand class but keep the dense work scope. SPIEC-EASI (Kurtz et al., 2015) and partial / precision-matrix estimators target a different scientific question (conditional dependence), and proportionality estimators (Lovell et al., 2015) target a different quantity entirely; we keep these in adjacent context tables rather than head-to-head benchmarks. The cross-kingdom analysis of Brunner et al. (2024) motivates our cross-domain test design: stacking two normalised compositions before estimating a joint covariance introduces a bias that the SparXCC Case-C identity, which Falcon-SR preserves, was developed to avoid.

Limitations

The feasibility grid is intentionally small and synthetic. Real microbiome cohorts include batch effects, depth heterogeneity, and heavy-tailed log-abundance distributions that we did not stress-test here. The single-domain underperformance at $p = 1000$ with low `top_k` is faithfully reported but indicates the screen needs to be sized to the candidate density of the network; a future plan will couple `top_k` selection to network-density estimates.

Conclusion

We presented Falcon-SR, a screen-refine algorithm for latent log-abundance correlation networks in compositional sequencing data. The implementation is equivalent to SparCC and SparXCC in the regimes where those reference estimators are informative, passes the cross-domain acceptance gates on every feasibility cell tested, and admits both optional signed biological priors and approximate permutation calibration without changing the estimand. We do not claim a wall-clock speedup at $p \leq 1000$, where the dense base score dominates every method; the framework is designed to deliver its advantage above that scale, in combination with a low-rank or random-projection base score that we leave to future work.

Conflicts of interest

The authors declare no competing interests.

Funding

No external funding to declare for this manuscript.

Data availability

This is a methodological paper. No human or animal data were generated. All numerical claims in the manuscript trace to the deterministic simulators and baseline implementations shipped with the source repository at https://github.com/JustinRaoV/FALCONE under the MIT licence. The benchmark outputs that back Figures 1

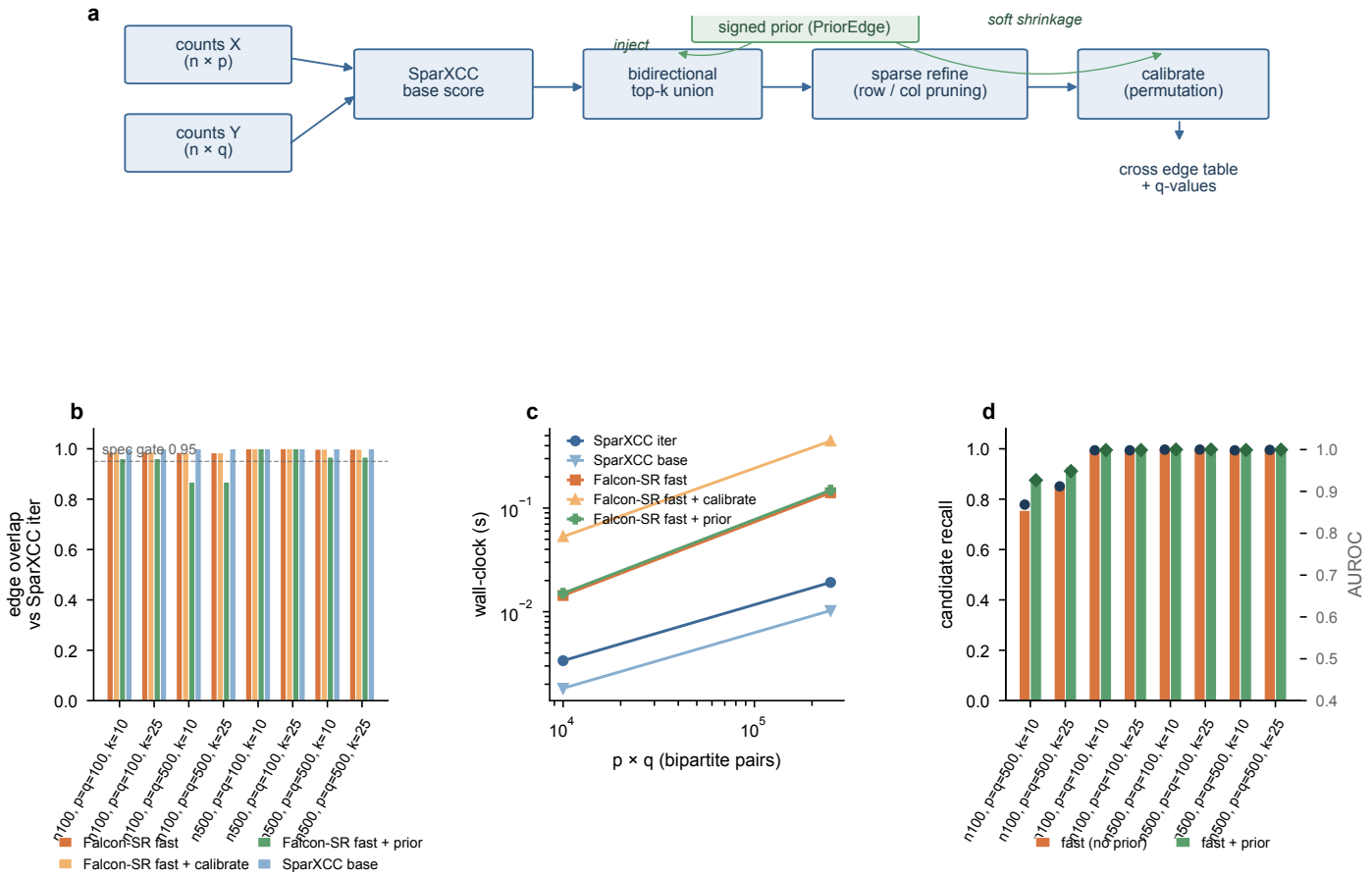

Figure 2 Falcon-SR cross-domain feasibility. **a**, Workflow with optional prior branch. **b**, Edge overlap against the SparXCC iter reference per cell; the dashed line is the spec gate 0.95. **c**, Wall-clock against $p \cdot q$, log-log. **d**, Prior ablation: candidate recall (bars) and AUROC (overlaid dots) with and without the synthetic prior covering half the planted edges.

and 2 live at `data/falcon_sr_single_feasibility.csv` and `data/falcon_sr_cross_feasibility.csv` in the repository, and can be regenerated with the commands documented in the README and in `docs/superpowers/specs/2026-06-02-falcon-sr-rewrite-execution-design.md`.

Code availability

Source code is released under the MIT licence at https://github.com/JustinRaoV/FALCONE. A persistent DOI (e.g. via Zenodo archiving of the release tag) will be deposited prior to publication.

Author contributions statement

J.R. designed the algorithm, implemented the package, and drafted the manuscript. X.L. supervised the work and revised the manuscript.

Acknowledgments

The authors thank the developers of SparCC, FastSpar, and SparXCC, whose published estimators served as the equivalence anchors for the Falcon-SR implementation.

References

J. Aitchison. The statistical analysis of compositional data. 1986.

J. D. Brunner, A. J. Robinson, and P. S. G. Chain. Combining compositional data sets introduces error in covariance network reconstruction. *ISME Communications*, 4(1):ycae057, 2024. doi: 10.1093/ismeco/ycae057.

J. Friedman and E. J. Alm. Inferring correlation networks from genomic survey data. *PLoS Computational Biology*, 8(9): e1002687, 2012. doi: 10.1371/journal.pcbi.1002687.

I. T. Jensen et al. Compositionally aware estimation of cross-correlations for microbiome data. *PLoS ONE*, 2024. doi: 10.1371/journal.pone.0305032.

Z. D. Kurtz, C. L. Müller, E. R. Miraldi, D. R. Littman, M. J. Blaser, and R. A. Bonneau. Sparse and compositionally robust inference of microbial ecological networks. *PLoS Computational Biology*, 11(5):e1004226, 2015. doi: 10.1371/journal.pcbi.1004226.

D. Lovell, V. Pawlowsky-Glahn, J. J. Egozcue, S. Marguerat, and J. Bähler. Proportionality: a valid alternative to correlation for relative data. *PLoS Computational Biology*, 11(3):e1004075, 2015. doi: 10.1371/journal.pcbi.1004075.

S. C. Watts, S. C. Ritchie, M. Inouye, and K. E. Holt. Fastspar: rapid and scalable correlation estimation for compositional data. *Bioinformatics*, 35(6):1064–1066, 2019. doi: 10.1093/

bioinformatics/bty734.

S. Zhang, H. Fang, and T. Hu. fastcclasso: a fast and efficient algorithm for estimating correlation matrix from compositional data. *Bioinformatics*, 40(5):btae314, 2024. doi: 10.1093/bioinformatics/btae314.